

Lista básica de consultas esenciales en PostgreSQL

Usuarios (roles con LOGIN)

- **Usuarios o roles con LOGIN**

sql

```
SELECT rolname
FROM pg_roles
WHERE rolcanlogin = true;
```

- Todos los usuarios o roles

\du

- **Contar usuarios**

sql

```
SELECT COUNT(*) AS total_usuarios
FROM pg_roles
WHERE rolcanlogin = true;
```

Roles (todos los roles del sistema)

- **Listar roles**

sql

```
SELECT rolname
FROM pg_roles;
```

- **Contar roles**

sql

```
SELECT COUNT(*) AS total_roles
FROM pg_roles;
```

Roles asignados a un usuario

- **Roles de un usuario**

sql

```
SELECT r.rolname AS rol
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
```

```
JOIN pg_roles u ON am.member = u.oid
WHERE u.rolname = 'maria';
```

Usuarios que pertenecen a un rol

- **Usuarios del rol vendedor**

sql

```
SELECT u.rolname AS usuario
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
JOIN pg_roles u ON am.member = u.oid
WHERE r.rolname = 'vendedor';
```

- **Contar usuarios en un rol**

sql

```
SELECT COUNT(*) AS total
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
WHERE r.rolname = 'vendedor';
```

- **Revocar un RLS o Rol de Grupo (Quitar herencia)**

```
REVOKE nombre_del_rol FROM nombre_usuario;
```

- Ejemplo real: Quitarle el rol de gerente a Sofía

```
REVOKE gerente FROM sofia_gerente;
```

Rol activo en este momento

- **Rol activo**

sql

```
SELECT session_user, current_user;
```

Esquemas

- **Listar esquemas**

sql

```
SELECT nsname AS esquema, pg_get_userbyid(nspowner) AS propietario
FROM pg_namespace
WHERE nsname NOT LIKE 'pg_%' AND nsname <> 'information_schema';
```

- \dn

- **Contar esquemas**

sql

```
SELECT COUNT(*) AS total_esquemas
FROM pg_namespace
WHERE nspname NOT LIKE 'pg_%' AND nspname <> 'information_schema';
```

Tablas por esquema

- **Listar tablas por esquema**

sql

```
SELECT tablename
FROM pg_tables
WHERE schemaname = 'ventas';
```

- \dt ventas.*;

- **Contar tablas por esquema**

sql

```
SELECT COUNT(*) AS total_tablas
FROM pg_tables
WHERE schemaname = 'ventas';
```

Secuencias por esquema

- **Listar secuencias**

sql

```
SELECT sequencename
FROM pg_sequences
WHERE schemaname = 'ventas';
```

- **Contar secuencias**

sql

```
SELECT COUNT(*) AS total_secuencias
FROM pg_sequences
WHERE schemaname = 'ventas';
```

Funciones por esquema

- **Listar funciones**

sql

```
SELECT proname
FROM pg_proc p
```

```
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname = 'ventas';
```

- \df

- listar funciones de un esquema (ventas,stock ...etc)

```
\df *.stock
```

- ver el codigo de una funcion :

```
\sf stock.reducir_stock
```

- **Contar funciones**

```
sql
```

```
SELECT COUNT(*) AS total_funciones
FROM pg_proc p
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname = 'ventas';
```

Permisos de una tabla

- **Ver permisos de una tabla**

```
sql
```

```
\z ventas.pedidos
```

Sí. Si quieres hacerlo **solo mediante SQL**, sin \du, puedes consultar pg_roles.

Para un rol concreto:

```
SELECT
    rolname,
    rolsuper,
    rolinherit,
    rolcreatorole,
    rolcreatedb,
    rolcanlogin,
    rolreplication,
```

```
rolbypassrls
FROM pg_roles
WHERE rolname = 'vioma';
```

Por ejemplo, el resultado podría ser:

```
rolname | rolsuper | rolinherit | rolcreaterole | rolcreatedb | rolcanlogin |
rolreplication | rolbypassrls
-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
vioma   | false    | true       | false          | false        | true         |
false   |           | false      |                 |               |              |
```

Qué significa cada atributo

Atributo	Significado
rolsuper	Es superusuario
rolinherit	Hereda automáticamente los privilegios de los roles de los que es miembro
rolcreaterole	Puede crear/modificar roles
rolcreatedb	Puede crear bases de datos
rolcanlogin	Puede iniciar sesión
rolreplication	Puede realizar conexiones de replicación
rolbypassrls	Puede saltarse Row-Level Security (RLS)

También puedes sacar **todos los atributos disponibles del rol**:

```
SELECT *
FROM pg_roles
WHERE rolname = 'vioma';
```

Y si quieres consultar **todos los roles del servidor**:

```
SELECT
    rolname,
    rolsuper,
    rolinherit,
    rolcreaterole,
    rolcreatedb,
    rolcanlogin,
    rolreplication,
    rolbypassrls
FROM pg_roles
ORDER BY rolname;
```

Si estás aprendiendo administración de PostgreSQL, `pg_roles` es una de las vistas del catálogo que merece la pena tener muy a mano.

y como sabemos si un rol puede hacer insert, select, delete etc?

Exacto. Aquí hay que distinguir **atributos del rol** (SUPERUSER, CREATEDB, etc.) de los **privilegios sobre objetos** (SELECT, INSERT, UPDATE, DELETE).

1. Ver los privilegios de un rol sobre una tabla

La forma más directa mediante SQL es:

```
SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'vioma'
ORDER BY table_name, privilege_type;
```

Te podría devolver:

grantee	table_schema	table_name	privilege_type
vioma	public	productos	DELETE
vioma	public	productos	INSERT
vioma	public	productos	SELECT
vioma	public	productos	UPDATE

Por tanto, **vioma** puede hacer las cuatro operaciones sobre **productos**.

2. Comprobar una tabla concreta

También puedes preguntarle directamente a PostgreSQL:

```
SELECT
    has_table_privilege('vioma', 'public.productos', 'SELECT') AS select,
    has_table_privilege('vioma', 'public.productos', 'INSERT') AS insert,
    has_table_privilege('vioma', 'public.productos', 'UPDATE') AS update,
    has_table_privilege('vioma', 'public.productos', 'DELETE') AS delete;
```

Resultado:

select	insert	update	delete
true	true	true	false

Esto significa:

- SELECT
- INSERT
- UPDATE
- DELETE

Esta segunda forma es especialmente buena, porque `has_table_privilege()` tiene en cuenta los privilegios que el usuario obtiene **directamente, mediante otros roles y por otras reglas de PostgreSQL**.

3. Para todas las tablas de una base de datos

Puedes hacer algo muy práctico:

```

SELECT
    table_schema,
    table_name,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'SELECT') AS
can_select,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'INSERT') AS
can_insert,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'UPDATE') AS
can_update,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'DELETE') AS
can_delete
FROM information_schema.tables
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
ORDER BY table_schema, table_name;

```

Así tienes una especie de **matriz de permisos del rol**:

schema	table	can_select	can_insert	can_update	can_delete
public	productos	true	true	true	false
public	pedidos	true	false	false	false
public	usuarios	true	false	false	false

Para administración de PostgreSQL, yo me quedaría con esta idea:

pg_roles → qué es el rol.

pg_auth_members → de qué roles hereda.

has_*_privilege() → qué puede hacer realmente.

estoy como jose y quiero cambiarme al rol de juani

Para “cambiarte” al rol **juani**, Jose, hay dos formas posibles en PostgreSQL, y solo una funciona dependiendo de cómo hayas creado ese rol.

Aquí tienes la explicación directa y sin rodeos.

1) Si juani es un usuario con LOGIN

(Es decir, un rol que puede conectarse)

Entonces **NO puedes cambiarte a él dentro de la sesión**. PostgreSQL no permite “convertirte” en otro usuario con LOGIN.

En ese caso, solo puedes **reconectar**:

```
sql
```

```
\c tienda_electronica juani
```

o con contraseña:

```
sql
```

```
\c "dbname=tienda_electronica user=juani password=TU_PASSWORD"
```

2) Si juani es un rol sin LOGIN

(Es decir, un rol de permisos, como “vendedor”, “auditor”, etc.)

Entonces **sí puedes activarlo** dentro de tu sesión actual:

```
sql
```

```
SET ROLE juani;
```

Y para volver a tu usuario original:

```
sql
```

```
RESET ROLE;
```

¿Cómo saber si juani es usuario o rol?

Ejecuta:

```
sql
```

```
SELECT rolname, rolcanlogin  
FROM pg_roles  
WHERE rolname = 'juani';
```

- Si `rolcanlogin = true` → **es usuario**
- Si `rolcanlogin = false` → **es rol**

Resumen rápido

- Si **juani es usuario** → reconectar con `\c`
- Si **juani es rol** → activar con `SET ROLE juani;`

Comando para ver qué rol tienes activo ahora

```
sql
```

```
SELECT session_user, current_user;
```

- `session_user` → tú (jose)
- `current_user` → el rol activo (si hiciste `SET ROLE`)

POLITICAS.

• ver las políticas actuales

```
SELECT polycname, roles, cmd, permissive, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas'
AND tablename = 'pedidos';
```

• HACER QUE UN USUARIO (JUANI) PUEDA SER GERENTE Y VER TODOS LOS PEDIDOS.

Sql

```
SELECT polycname, roles, cmd, permissive, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas'
AND tablename = 'pedidos';
-----
```

```
"pedidos_gerente" "{gerente}" "SELECT" "RESTRICTIVE" "true"
"pedidos_por_usuario" "{public}" "SELECT" "PERMISSIVE" "(usuario =
CURRENT_USER)"
```

```
-- 1. Le quitamos el rol de cliente a Juani
REVOKE vendedor FROM juani;
```

```
-- 2. Le asignamos el nuevo rol de gerente
GRANT gerente TO juani;
```

1. El peligro del tipo **RESTRICTIVE** (Restringido)

Tienes la política de gerentes configurada como **RESTRICTIVE**, mientras que la de usuarios normales está como **PERMISSIVE**.

En PostgreSQL, las políticas funcionan así:

- Las políticas **PERMISSIVE** (Permisivas) se suman con un **OR**.
- Las políticas **RESTRICTIVE** (Restrictivas) se aplican después como un filtro obligatorio usando un **AND**.

Al poner la de gerente como **RESTRICTIVE**, le estás diciendo a PostgreSQL: *"No importa qué diga cualquier otra política, **obligatoriamente** se tiene que cumplir la condición de gerente"*. Como Juani es gerente, pasa ese filtro, pero luego PostgreSQL lo evalúa contra la política permisiva (`usuario = CURRENT_USER`). Como ambas se terminan cruzando de forma estricta por culpa del diseño, Juani queda atrapado viendo solo lo suyo.

2. La condición (`usuario = CURRENT_USER`)

Tu política permisiva dice exactamente `usuario = CURRENT_USER`. Si Juani está conectado como `juani`, el motor solo le va a devolver las filas donde la columna `usuario` sea exactamente igual a `'juani'`.

Cómo solucionarlo ahora mismo

Para que Juani (y cualquier gerente) pueda ver todos los pedidos, **todas las políticas de consulta (SELECT) deben ser PERMISSIVE**. De esa forma, PostgreSQL dirá: *"O ves tus propios pedidos OR eres gerente y ves todo"*.

Ejecuta este código como administrador para borrar esas políticas y crearlas correctamente:

SQL

```
-- 1. Borramos las políticas problemáticas que tienes actualmente
DROP POLICY IF EXISTS pedidos_gerente ON ventas.pedidos;
DROP POLICY IF EXISTS pedidos_por_usuario ON ventas.pedidos;

-- 2. Creamos la política para usuarios normales (Permisiva)
CREATE POLICY pedidos_por_usuario
ON ventas.pedidos
AS PERMISSIVE
FOR SELECT
TO public
USING (usuario = CURRENT_USER);

-- 3. Creamos la política para gerentes (¡También Permisiva!)
CREATE POLICY pedidos_gerente
ON ventas.pedidos
AS PERMISSIVE
FOR SELECT
TO gerente
USING (true); -- "true" significa que el rol gerente tiene permitido ver TODAS
las filas

SELECT policyname, roles, cmd, permissive, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas'
AND tablename = 'pedidos';
-----
"pedidos_por_usuario" "{public}" "SELECT" "PERMISSIVE" "(usuario =
CURRENT_USER)"

"pedidos_gerente" "{gerente}" "SELECT" "PERMISSIVE" "true"
```

¿Por qué esto sí va a funcionar?

Al ser ambas políticas **PERMISSIVE**, PostgreSQL las unirá con un **OR**.

Cuando Juani consulte la tabla:

1. ¿El pedido es de Juani? (Política 1).
2. ¿Juani pertenece al rol **gerente**? **Sí**, por lo tanto la Política 2 devuelve **true** para **todas** las filas de la tabla.
3. Como **Falso OR True = True**, ¡Juani podrá ver el listado completo de pedidos automáticamente!

TRIGERS-

. ver todos los triggers

```
SELECT
    tname AS nombre_trigger,
    relname AS tabla_asociada,
    pg_get_triggerdef(pg_trigger.oid) AS
codigo_completo_del_trigger
FROM pg_trigger
JOIN pg_class ON pg_trigger.tgrelid = pg_class.oid
JOIN pg_namespace ON pg_class.relnamespace = pg_namespace.oid
WHERE tgisinternal = false
    AND nspname NOT IN ('pg_catalog', 'information_schema');
```

\dy

. ver triggers de una tabla:

\d ventas.pedidos

. Script para ver qué ver el trigger junto con su función

```
SELECT
    tbl.relname AS tabla,
    trg.tname AS nombre_trigger,
    -- Aquí vemos la definición del disparador (CREATE TRIGGER...)
    pg_get_triggerdef(trg.oid) AS definicion_del_trigger,
    -- Aquí vemos el código interno de la función que ejecuta
    p.prosrc AS codigo_interno_funcion
FROM pg_trigger trg
JOIN pg_class tbl ON trg.tgrelid = tbl.oid
JOIN pg_proc p ON trg.tgfoid = p.oid
WHERE trg.tgisinternal = false;
```